Finite State Machine: Traffic Light Controller for a 2-Way Intersection

ECE 2029: Project Report

Noah Herbek, Mathias Torres, Spring 2025

Introduction:

Many people around the world drive cars or use public transportation to get from one point to another. Without the presence of properly functioning traffic lights, driving would be extremely dangerous as the flow of traffic would not only be heavily congested and confusing, but also the safe travel of thousands of drivers would not be guaranteed, with no signals to stop or go, and instead relying on the trust and intuition of others. As a group, we realized how important traffic lights are, as they direct traffic efficiently, require little maintenance, and guarantee the previously mentioned safety of drivers. Thus, we decided to create an exemplary traffic light system at one of many intersections a driver could come across in their day-to-day life and travels, not only focusing on the looping and cycling of the red, yellow, and green lights, but also emphasizing the significance of sensors that allow the noted patterns in how traffic lights work to maximize safety and minimize traffic congestion.

Motivation:

Prior to beginning this project, we had various ideas for what types of finite state machines we wanted to design. While they were all interesting, we believed that designing an efficient traffic light system would be valuable in terms of teaching us about the importance of finite state machines and the eventual design of them, all the while, producing a system that is so commonly experienced in most people's day-to-day lives. Rather than being a system that changes states based on various inputs, we decided to make a system that has multiple states for just a few inputs, with its sequential logic being most apparent with the timed cycling of its various states.

Problem Statement:

This would be a Moore FSM to control traffic lights for a main road and a side road. The main road gets more green time by default, and the side road gets green only if a car is detected (via a sensor).

Some specifications of our design: The one input of our design would be the sensor detecting the cars from the side road to see if the light needs to be triggered or not, which can be represented by one binary bit (on or off). The two outputs would be the state of the two lights, which could be represented by two binary bits for the three states of the light (green, yellow, and red). The design will wait to sense a car,

and once it senses it, a timer will start to cycle both of the lights to allow the car to go. Once the lights have been cycled, the system will return back to the original state of waiting for a car and allowing the main road to have the green light. Between each state, there will be delays for proper timing of the light to ensure that no cars crash into each other and to have a smooth flow of cars. The inputs of the sensor do not matter between the light cycle because once the car is sensed in the original state, it will cycle through them all no matter what the sensor is sensing.

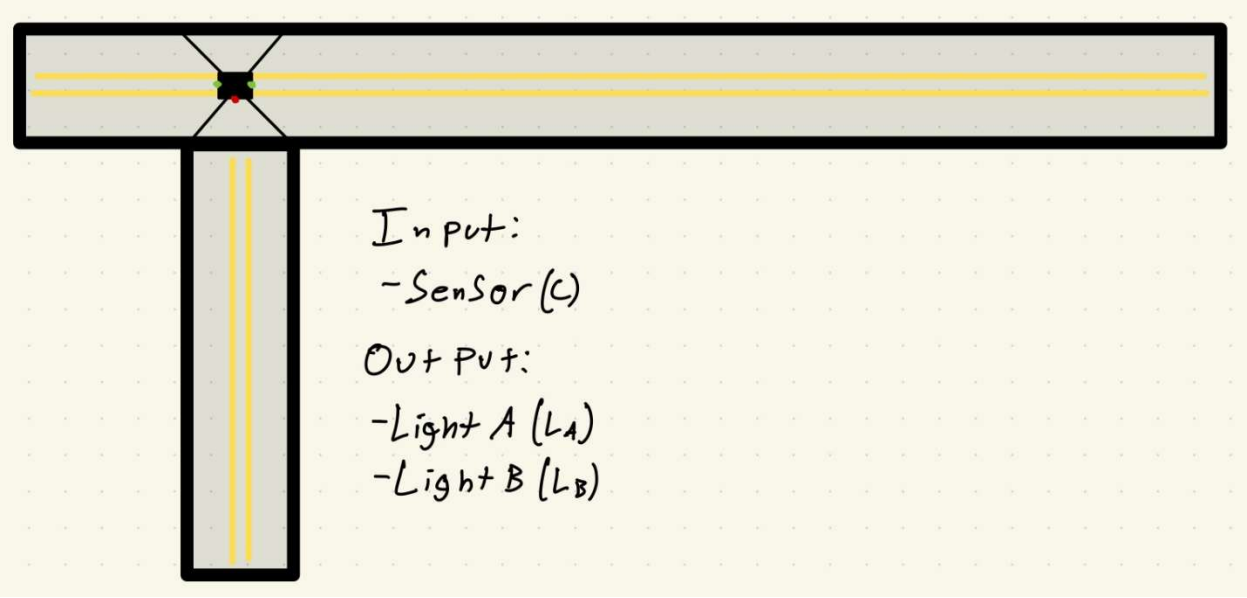

Figure 1. Visual diagram of the 2-way intersection, alongside the traffic light outputs and input sensor.

A possible graph of how the input and outputs could work is shown below:

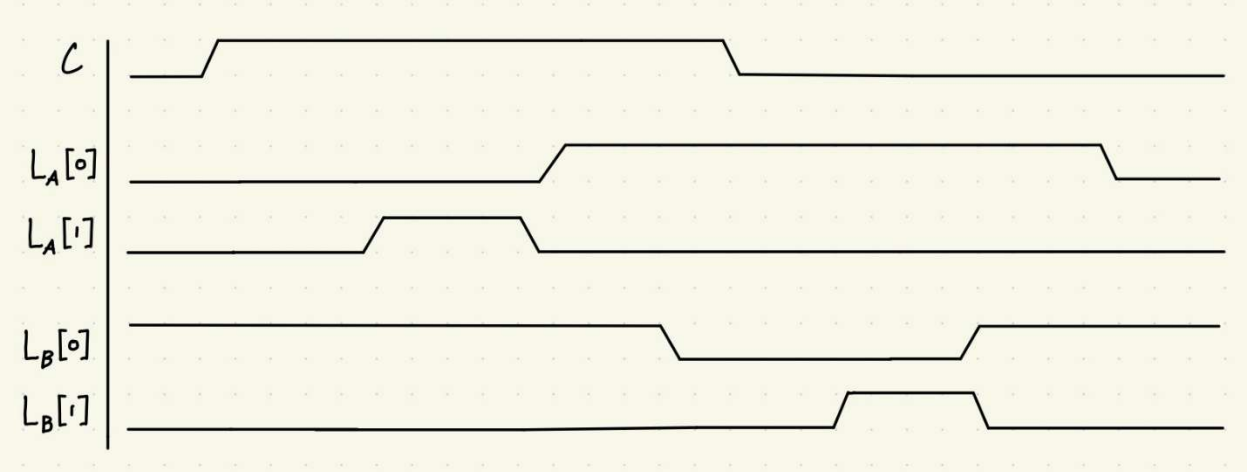

Figure 2. Above is a graph depicting how the main and side traffic lights, L_A and L_B respectively, change according to the input detected by the car detection sensor of on the side road.

Proposed Solution:

Due to the nature of the intersection where the traffic light system is being developed, the traffic light primarily remains green for the main road and red for the side road due to their respective road usage, with the main road being used more. As there's more traffic on the main road, keeping the traffic light in this initial state up until the side road sensor detects a car is efficient, as it will not unnecessarily stop cars on the main road, when there is a lack of cars on the side road. When a car is detected on the side road, the traffic light begins to cycle through its six states, stopping all traffic on the main road, and allowing traffic on the side road to proceed, with the input of the sensor essentially being unimportant to the cycling of the traffic light system. Once the side road traffic light turns red, the system returns back to its initial state until the sensor detects a car on the side road once more, cycling through in an efficient manner.

State Diagram:

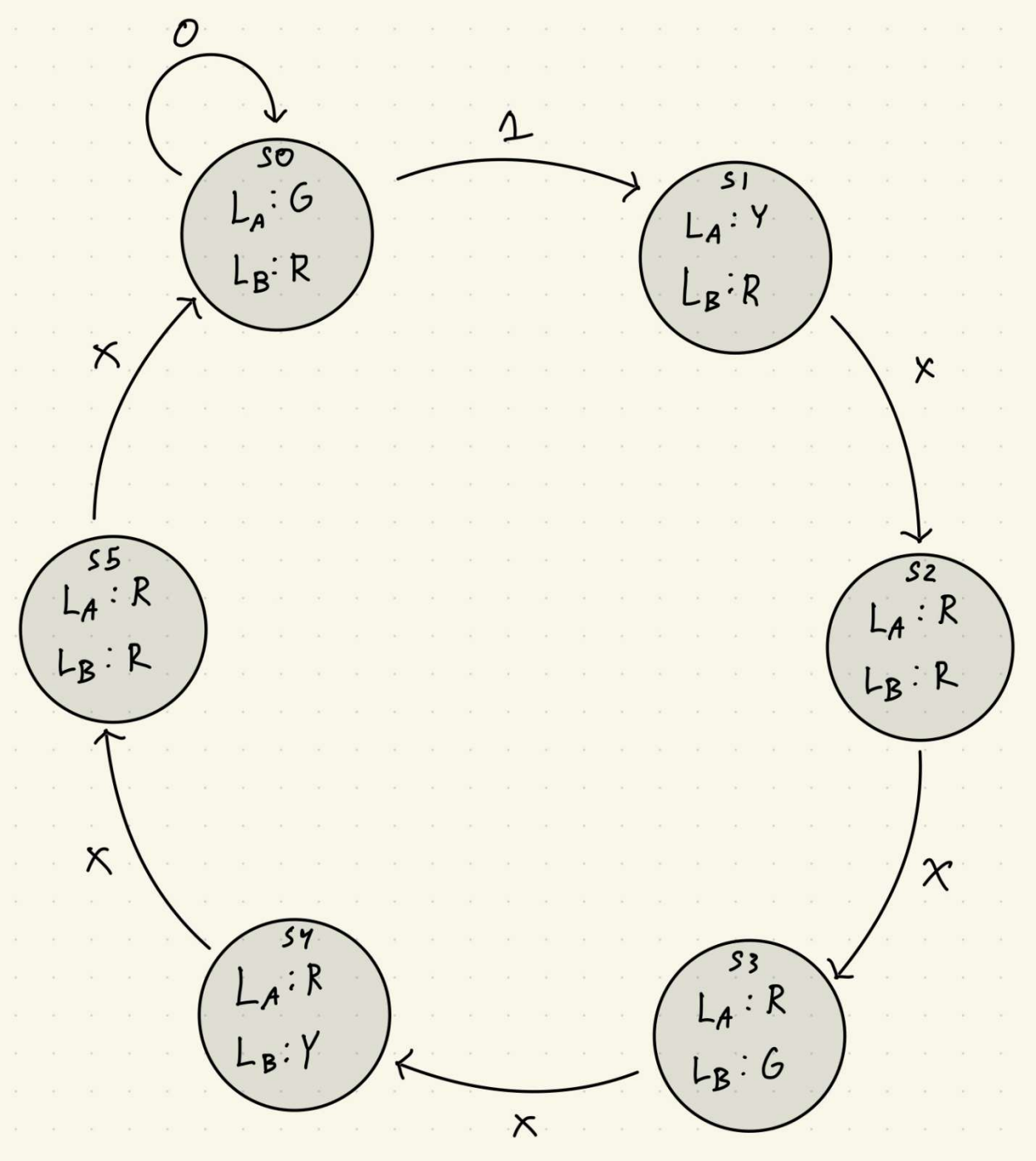

Figure 3. Above is a state diagram depicting the six states that the traffic light system has to go through to ensure safe and smooth passage of traffic.

State Assignments:

State	x	y	z
S0	0	0	0
S1	0	0	1
S2	0	1	0
S3	0	1	1
S4	1	0	0
S5	1	0	1

Table 1. Above is a state assignment table for this Moore's finite state machine.

State table:

Current State: X Y Z	Input: C	Next State: X+ Y+ Z+	Output: L_A	L_B
S0 0 0 0	0	S0 0 0 0	$G_{(00)}$	$R_{(10)}$
S0 0 0 0	1	S1 0 0 1	$G_{(00)}$	$R_{(10)}$
S1 0 0 1	X	S2 0 1 0	$Y_{(01)}$	$R_{(10)}$
S2 0 1 0	X	S3 0 1 1	$R_{(10)}$	$R_{(10)}$
S3 0 1 1	X	S4 1 0 0	$R_{(10)}$	$G_{(00)}$
S4 1 0 0	X	S5 1 0 1	$R_{(10)}$	$Y_{(01)}$
S5 1 0 1	X	S0 0 0 0	$R_{(10)}$	$R_{(10)}$

Table 2. Above is a state transition table of the traffic light Moore's finite state machine with an output table affected only by the current state.

K-Maps:

Input:

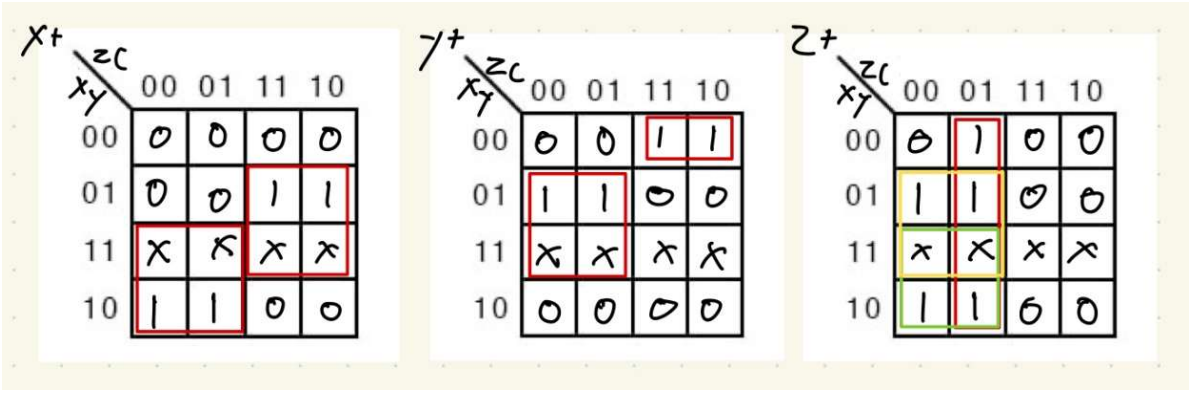

Figure 4. Above are the three k-maps used to find the Boolean equations for each of the three bits used to represent the six necessary states for the traffic light finite state machine.

Output:

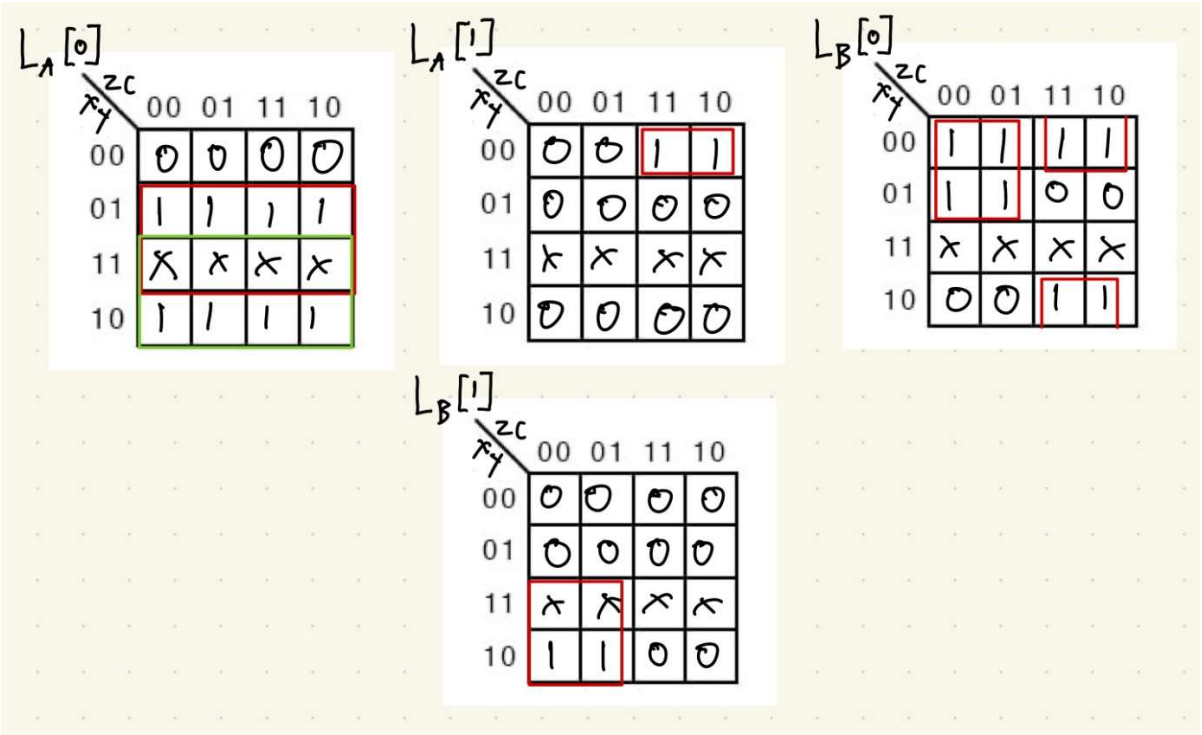

Figure 5. Above are the four k-maps used to find the two bits designated to the main and side traffic lights outputs, indicating the color they show in specific states.

Boolean Equations:

Input:

$$x^{+} = (xz') + (yz)$$

$$y^{+} = (x'y'z) + (yz')$$

$$z^{+} = C + (yz') + (xz')$$

Output:

$$L_A[0] = x + y$$

$$L_A[1] = (x'y'z)$$

$$L_B[0] = (y'z) + (x'z')$$

$$L_B[1] = (xz')$$

Implemented Solution:

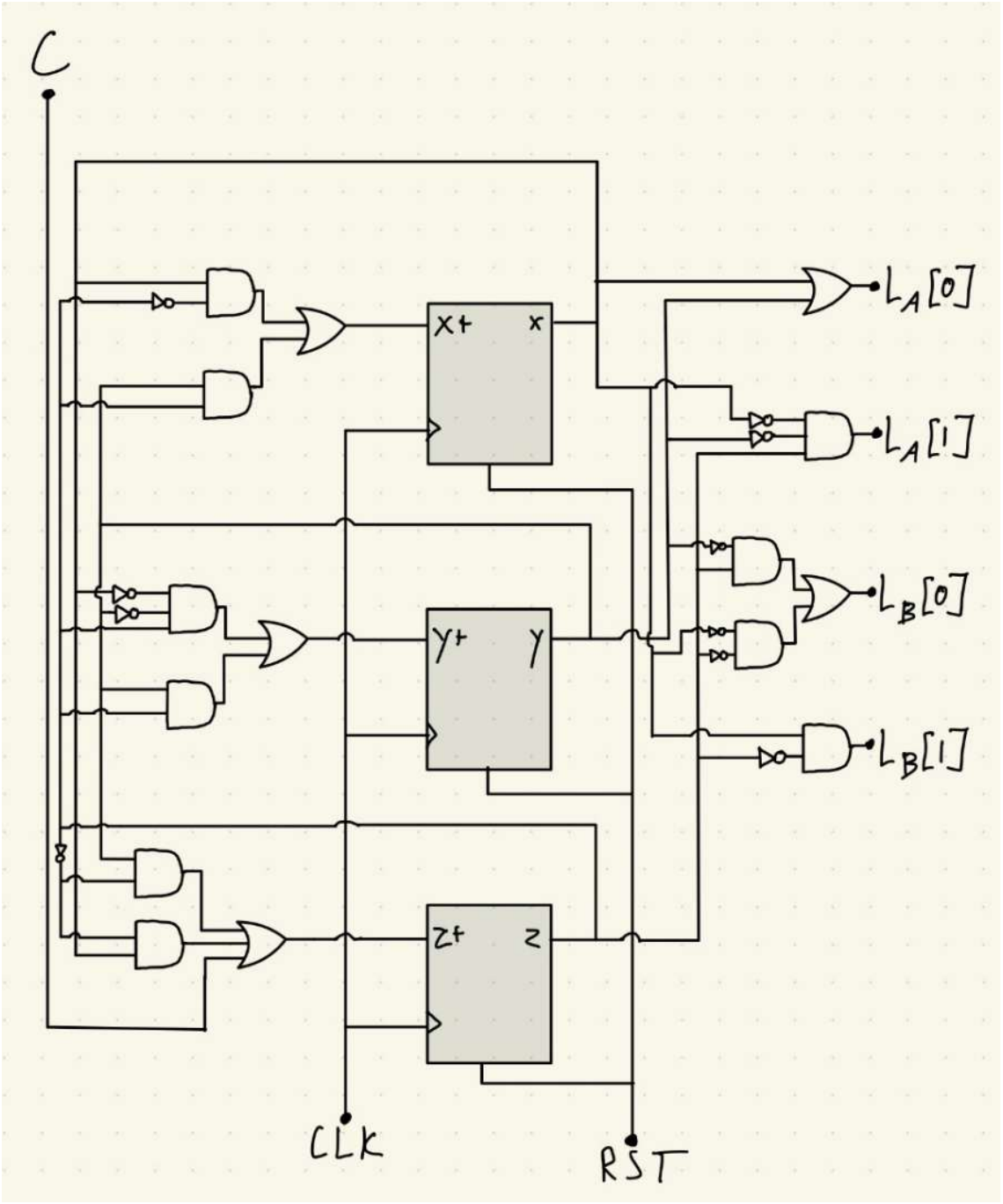

Figure 6. Above is a finite state machine schematic designed based on the input and output Boolean equations found from the k-maps to design an efficient traffic light system.

Verilog Implementation:

Design Code:

Figure 7. Above is the design source Verilog code utilized to make the traffic light finite state machine. [1] [2]

---

[1] ChipVerify, "Verilog FSM (Finite State Machine)," accessed [April 30, 2025], https://www.chipverify.com/verilog/verilog-fsm.

[2] ASIC-World, "Verilog Finite State Machine (FSM)," accessed [April 30, 2025], https://www.asic-world.com/tidbits/verilog_fsm.html.

The following design source code first begins by setting up the clk (clock), reset, and car_detected (sensor which detects cars on the side road) inputs, alongside the two-bit main and side traffic light outputs, represented by main_light and side_light respectively. Alongside a 3-bit state register, used to cycle through the six states of this system, the main_light and side_light outputs are also registers, allowing for the sequential logic of this system to work properly. To begin, the code establishes that when the reset input is detected as a 1, the initial state that the traffic lights should return to is at a point where the main traffic light is green, while the side traffic light is red, labeled s0. From that point onwards, case statements are used to cycle through each of the six states. The initial state, s0, is the only state in which the input detected from car_detected plays a significant role, as a 0 wouldn't change the initial state, but a 1 would lead to the proper cycling through each state automatically. Any other car_detected input read while the system is cycling is ignored and does not affect the function of the traffic lights to ensure the proper cycling of lights. The states change according to the rising edge of the clock input, or on the falling edge of the reset input if the reset were to be activated. After cycling through to the sixth, and final state, s5, where both the main and side traffic lights are red, the code establishes the first state, s0, as the default state, and thus the system returns to said state automatically.

Simulation Code:

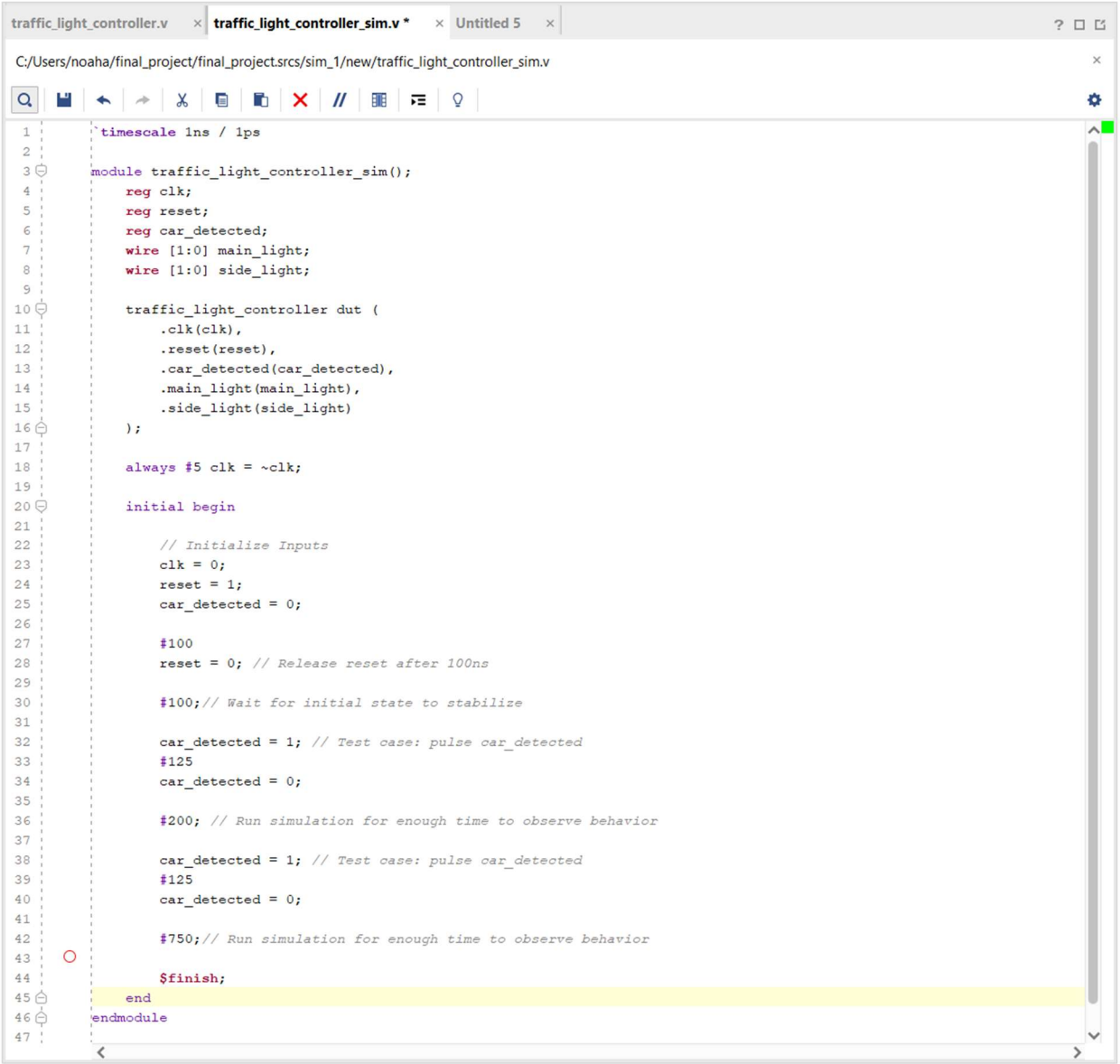

Figure 8. Above is the simulation source/testbench Verilog code utilized to test out various cases that could occur within the traffic light finite state machine.

For the testbench code, the clk, reset, and car_detected inputs from the traffic_light_controller module are placed in as registers, while the main_light and side_light outputs are placed in as two-bit wires. To begin, the timing of the clock is set to 5 nanoseconds. From there, the simulation is initialized at the default state of s0 after reading a 1 from the reset input. After a 100 nanosecond wait, a case where the car_detected input is 1 is tested, cycling through the six states of the finite state

machine over a span of about 325 nanoseconds, allowing for the proper observation time of the functionality of the traffic lights, and providing guaranteed safety for states to change without the danger of car accidents. After returning to the default state of s0, another cycling through the six states is tested out to ensure that the traffic lights only change states when a car is detected by the sensor on the side road, rather than always changing according to simply just the clock.

Simulation:

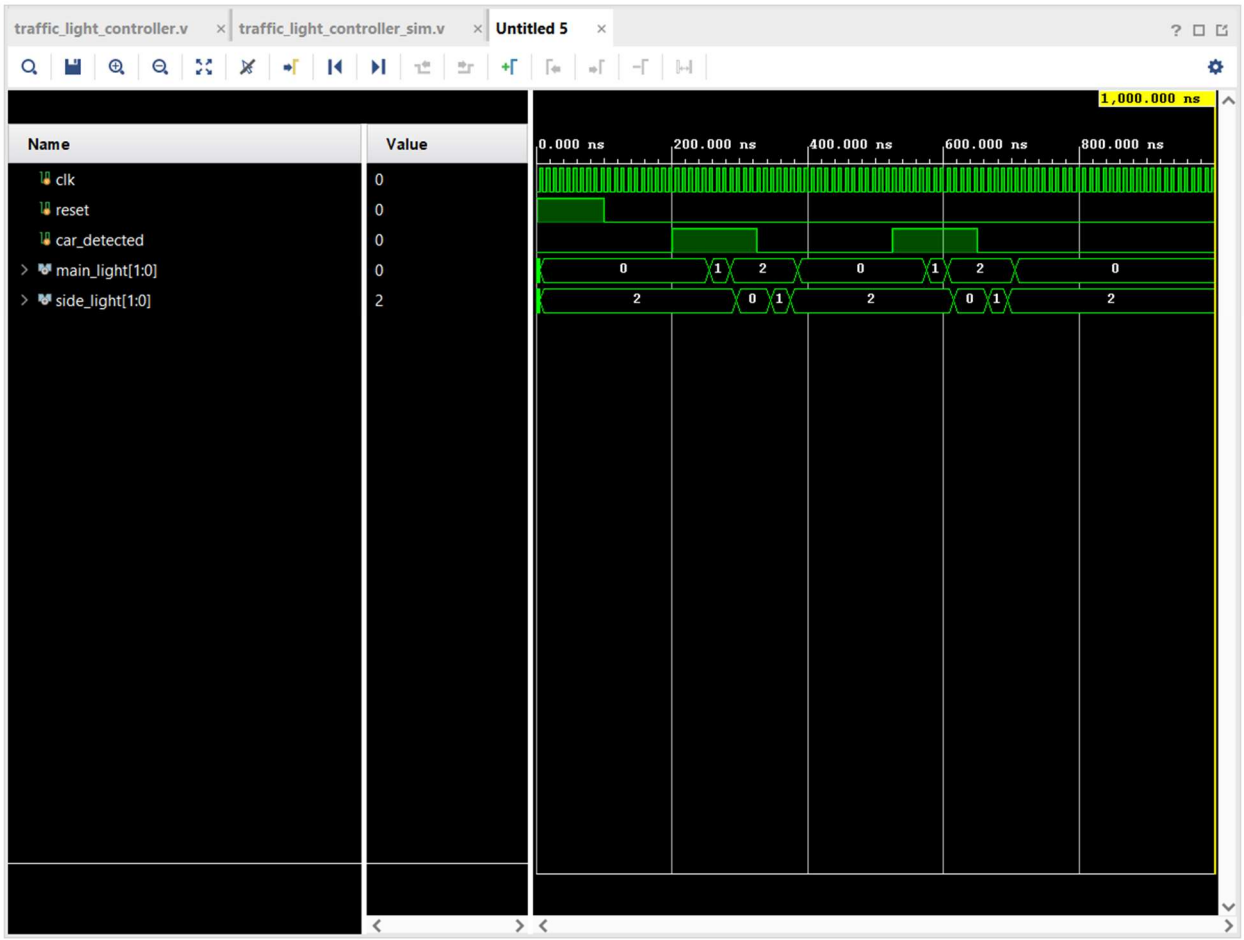

Figure 9. Above are the simulation results for the traffic light finite state machine, showing how the system cycles through all of its states when it detects a car on the side road and then remains in its default, initial state otherwise, activating based on the rising edge of the clock.

The simulation above depicts how the clock, reset, and car_detected inputs affect the functionality of the traffic light finite state machine and the main_light and side_light outputs. The first test case considered was an initial state where the reset was activated, changing both lights to their initial state where the main_light is green, and the side_light is red (with 00 or 0 representing a green light, 01 or 1 representing a yellow light, and 10 or 2 representing a red light). It can be seen how a car_detected input of 1 leads to the cycling of the main_light and side_light outputs, always changing on the rising edge of the clock input, until it returns once more to s0. From there, the test case where car_detected is activated is tested once more, and the system cycles through once more, before returning to the default state, where it is to stay until car_detected is detected as a 1 input yet again.

Elaborated Design:

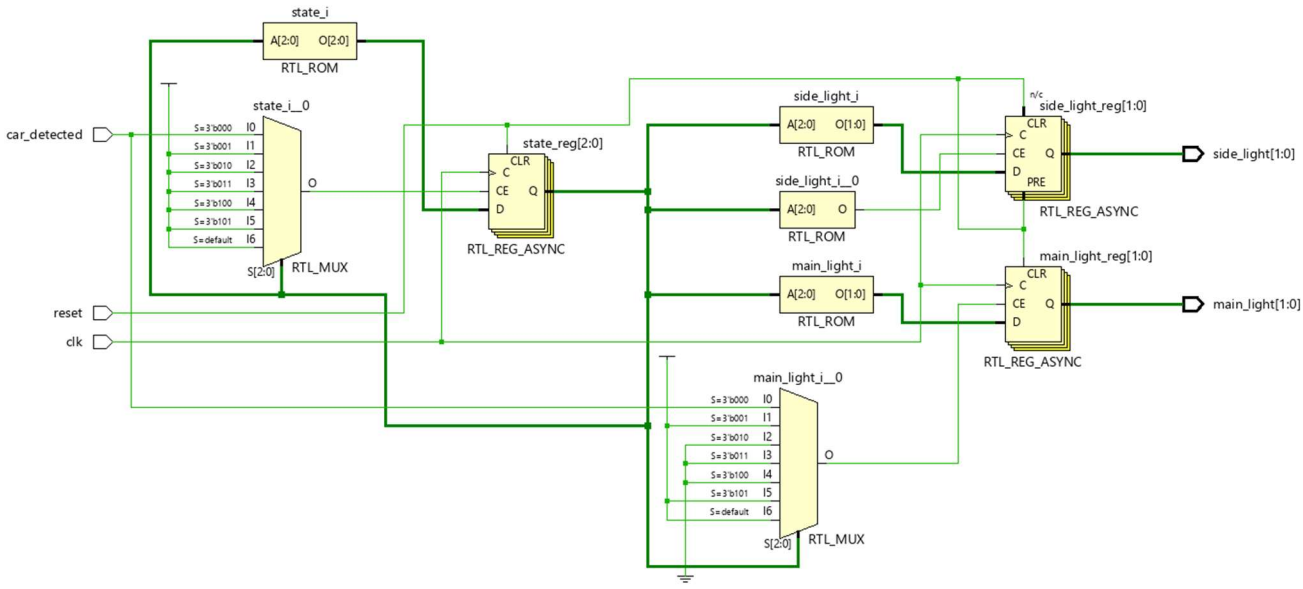

Figure 10. Above is the elaborated design of the traffic light finite state machine that achieves the same results as figure 6.

Conclusion:

Overall, the simulation results indicate that the manner in which this state machine was designed is very efficient as it allows through the cycling of traffic lights at the detection of a car on a side road, all the while ensuring that traffic flows smoothly and safely. Ultimately, while the implemented solution design

contained 25 basic logic gates in addition to three D flip-flops, the elaborated design generated through the Verilog code indicates that two multiplexers could be used in addition to the three registers and the respective inputs and outputs to achieve the same results. Regardless, the finite state machine is believed to be as efficient as it can be, as the 6 states utilized cover each possible, and logical, combination of traffic lights in a 2-way intersection, to guarantee a safe and smooth passage of traffic from either road.

This project demonstrated how the number of inputs in a finite state machine, besides the clock and reset inputs, doesn't necessarily impact the number of states. Multiple states and outputs can be achieved with just one input. To emphasize said point further, an input may only be significant for one state but lead to an important cycling of states, where the inputs no longer matter, allowing for the realization that finite state machines are dictated by their states, rather than the number of inputs. In addition, the elaborated design reveals how the most efficient solutions to a finite state machine can be designed differently, all the while producing the same results overall, meaning that the costs of designs can potentially be reduced with a more advanced design, though the usage of logic gates in the initial design process is important to understand what combinations of inputs and bits lead to specific outputs and states.

In terms of future work or extensions that could be worked upon the foundation set up by this finite state machine, multiple paths could be taken. Amongst those paths, a case where a pedestrian crossing light exists could be integrated into this situation, which would turn all traffic lights to red and a pedestrian crossing light to indicate safe passage of pedestrians (or turn green, depending on how the light is designed). Another extension that could be considered is the presence of a 4-way intersection, rather than a 2-way intersection, where an additional green arrow light could be added alongside the solid-colored lights to guarantee left-turning cars the right of way, meaning that traffic lights would have to be set up according to their lanes and respective roads, which would definitely be more complicated, as only four of potentially eight traffic lights would have a green arrow lights. Regardless, as mentioned before, the efficiency with which this system was designed could be set up as a foundation for such a case, and any other cases revolving around the cycling of traffic lights and any other road signals at an intersection.

References:

2025. *How to write FSM in Verilog?* January 7. https://www.asic-world.com/tidbits/verilog_fsm.html.

2023. *Verilog FSM.* September 8. https://www.chipverify.com/verilog/verilog-fsm.

GitHub Repository:

https://github.com/noah-herbek/Traffic-Light-Controller-for-a-2-Way-Intersection